



Lecture 3a: Data Formatting & Model Reliability

Splits, Cross-Validation, and Leakage

Omkar Bait (CosmicAI Fellow)
NRAO, Charlottesville

CosmicAI Summer School

Why Split Your Data?

A model that memorizes the training set is worthless on new galaxies.

The trap: training accuracy

A flexible model can fit **any** training set perfectly — even pure noise.

Training accuracy says **nothing** about how the model will perform on tomorrow's data.

We need a **held-out** sample the model has never seen.

Two failure modes

Overfitting: model fits noise as if it were signal.

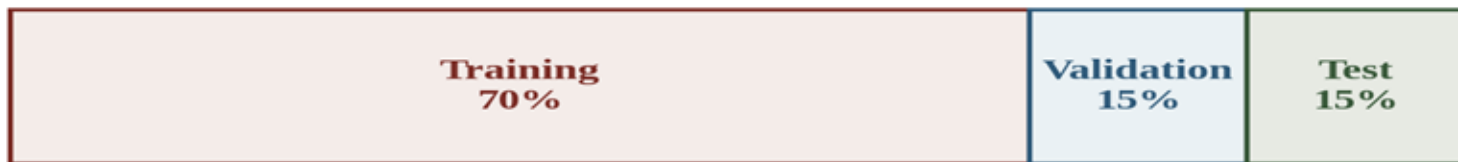
Symptom: $\text{train} \gg \text{test accuracy}$.

Underfitting: model is too simple to capture the structure.

Symptom: $\text{train} \approx \text{test}$, both bad.

The fix: split the data into Train, Validation, and Test — and use each for one job only.

Train / Validation / Test



fit parameters

tune hp

final score

Training

~70%

Fit the model parameters. Loss is minimized here.

Validation

~15%

Tune hyperparameters (tree depth, k in KNN). Pick the best model.

Test

~15%

Touch only once, at the very end. Reported number = your final score.

Iron rule: the Test set is a one-shot. If you peek and re-tune, it's no longer a test.

Hold-out Validation

Recipe:

- Randomly split into train / val (e.g. 80/20).
- Train the model on the train set.
- Compute the metric on the validation set.
- Repeat for every hyperparameter choice.

Pros: fast — only one train per config.

Cons: the score depends on which random split you got.

```
from sklearn.model_selection import
train_test_split

# stratify=y when classifying
X_train, X_val, y_train, y_val = (
    train_test_split(
        X, y,
        test_size=0.2,
        random_state=42,
        stratify=y,
    )
)
```

Watch out: one bad split can mislead. Use a fixed `random_state` for reproducibility, then validate with k-fold (next slide).

K-Fold Cross-Validation

	<i>chunk 1</i>	<i>chunk 2</i>	<i>chunk 3</i>	<i>chunk 4</i>	<i>chunk 5</i>	
fold 1	val	train	train	train	train	score = mean of fold-1 ... fold-k
fold 2	train	val	train	train	train	
fold 3	train	train	val	train	train	
fold 4	train	train	train	val	train	
fold 5	train	train	train	train	val	

How it works:

Split data into **k** equal chunks (typical $k = 5$ or 10).
Hold one chunk out as validation, train on the rest.
Repeat for every chunk. Average the k scores.

Why bother:

Every example serves as validation exactly once.
More robust estimate (mean $\pm$ std across folds).
Especially valuable for small astro samples.

```
KFold(n_splits=5, shuffle=True, random_state=42)
```

Regression Metrics — Continuous y

When the target is a continuous number, you have a standard toolbox of 4 metrics.

MSE

$$\text{MSE} = (1/N) \sum (y_i - \hat{y}_i)^2$$

Squared units. Sensitive to outliers. Mathematically convenient (differentiable).

RMSE

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Same units as y. Default for reporting (interpretable).

MAE

$$\text{MAE} = (1/N) \sum |y_i - \hat{y}_i|$$

Same units as y. Robust to outliers (no squaring).

Don't forget BIAS: $\langle \hat{y} - y \rangle$ — tells you if the model systematically over- or under-predicts. **Photo-z (Lec 4):**
 $\text{RMSE} = 0.025$, $\text{bias} = +0.0001 \rightarrow$ accurate and unbiased.

Classification Metrics — Discrete y

Class labels need a richer toolbox. Start with the confusion matrix.

Confusion matrix

	Predicted YES	Predicted NO
Actual YES	TP	FN
Actual NO	FP	TN

Metrics

Accuracy

$$(TP + TN) / \text{total}$$

fraction correct overall

Precision (Purity)

$$TP / (TP + FP)$$

of YES calls, how many right?

Recall (Completeness)

$$TP / (TP + FN)$$

of true YES, how many caught?

F1

$$2 \cdot P \cdot R / (P + R)$$

harmonic mean of P & R

ROC-AUC: area under TPR-vs-FPR curve.
Threshold-free.

Use F1 or AUC when the rare class is what matters.

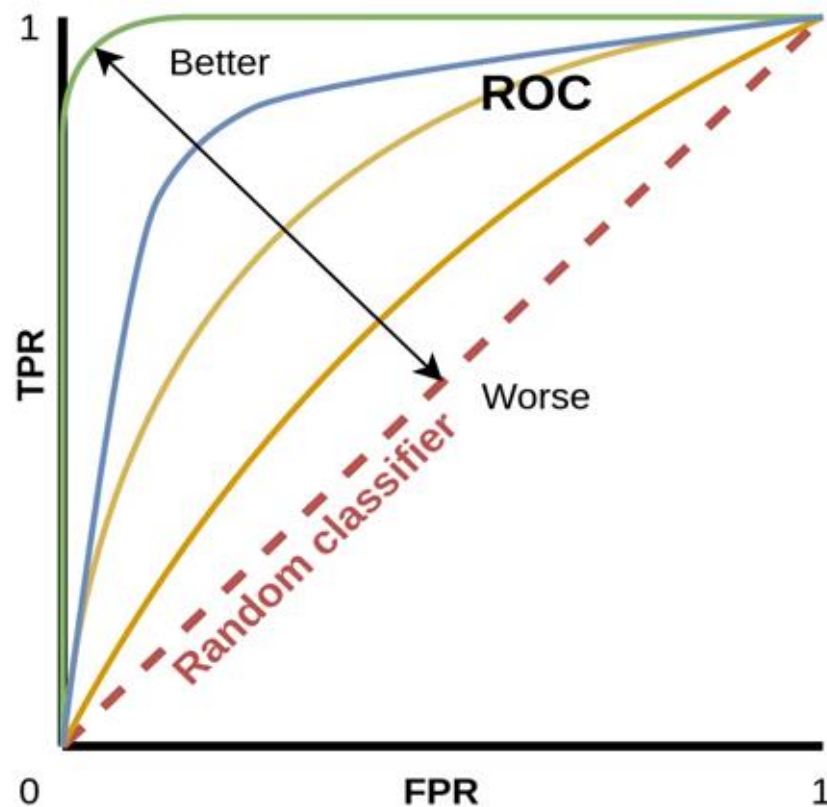
Classification Metrics — Discrete y

Class labels need a richer toolbox. Start with the confusion matrix

Confusion matrix

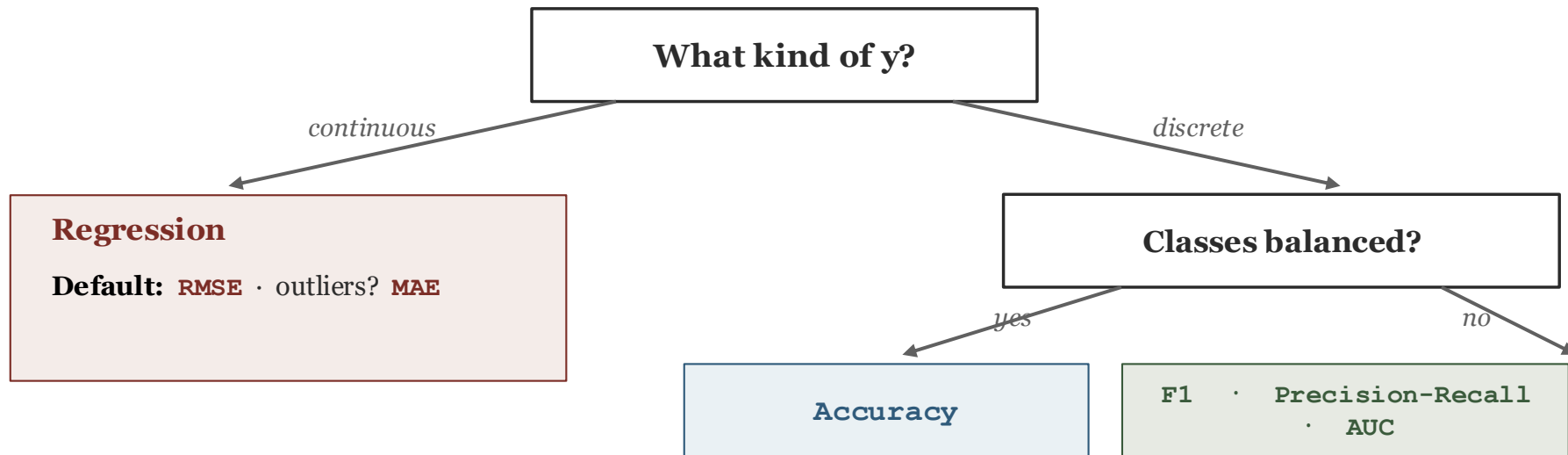
	Predicted YES	Predicted NO
Actual YES	TP	FN
Actual NO	FP	TN

Use F1 or AUC when the rare class is what matters.



Picking the Right Metric

Match the metric to the problem and to what matters scientifically.



Iron rule: always report **2–3 metrics**, never just one. A single number hides important behaviour (bias, recall on the rare class, outlier impact).

Quick Check – Data Preprocessing

QUIZ

Why must scalers (StandardScaler, MinMaxScaler) be fit on the training set only?

A

It's faster than fitting on all data

B

Otherwise test-set statistics leak into the training process

C

sklearn requires it for the API to work

D

Min and max values from the test set are unreliable

Pick A, B, C, or D – then advance.

Quick Check 1 — Answer

ANSWER

A

It's faster

Performance is not the reason — the leak is the reason.

B

Otherwise test stats leak in ✓

Data leakage: μ and σ from test would influence train.

C

sklearn requires it

No — sklearn lets you do anything; YOU enforce hygiene.

D

Min/max unreliable

True but secondary; leakage is the root reason.

Correct workflow: `scaler.fit_transform(X_train)` → then `scaler.transform(X_test)`. Inside a **Pipeline**, this happens automatically inside each CV fold.

Quick Check

QUIZ

You're classifying gravitational lenses. Only 1% of objects are actually lenses. Your model predicts "NOT a lens" for every example. What is its accuracy?

A

50%
— like a coin flip

B

99%
— but useless

C

1%
— gets all lenses wrong

D

Undefined
— model never said "yes"

Pick A, B, C, or D — then advance.

Quick Check — Answer

ANSWER

A

50%

Coin-flip would require 50/50 base rate, not 1/99.

B

99% — but useless ✓

99% of test cases ARE "not a lens" → they're all classified correctly.

C

1%

Confuses accuracy with recall. Recall on lenses = 0, but accuracy = 99.

D

Undefined

Accuracy is always defined as $(TP + TN) / \text{total}$.

Lesson: accuracy looks great here (99%) but the model is **scientifically useless**. For rare-class detection, look at **recall, F1, or AUC** — they expose the failure that accuracy hides.

Quick Check 1

QUIZ

*You're training a classifier to identify gravitational lenses (a rare class — only ~1% of your sample).
Which CV variant should you use?*

A

Plain KFold
— random splits

B

StratifiedKFold
— preserves class proportions

C

GroupKFold
— honors example groups

D

TimeSeriesSplit
— train on past only

Pick A, B, C, or D — then advance.

Quick Check 1 — Answer

ANSWER

A

Plain KFold

A random split may put 0 lenses in some folds — useless.

C

GroupKFold

'Group' means examples that share an entity (cluster, image).

B

StratifiedKFold



Each fold gets ~1% lenses — preserved class proportions.

D

TimeSeriesSplit

Use only when temporal ordering matters.

sklearn one-liner: `cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`
`scores = cross_val_score(model, X, y, cv=cv, scoring="f1")`

Labels — Ground Truth $\neq$ Truth

Your labels are estimates too. Garbage labels $\rightarrow$ garbage model.

Labeled data

Has y . Often expensive — spectroscopic redshift, hand-classified morphology, confirmed transient.

Unlabeled data

Just X . The bulk of any modern survey (photometric only). Used in semi-/self-supervised settings.

Noisy labels

Labels exist but are uncertain — photo- z estimates used as 'truth', crowd-sourced morphology, automated pipelines.

Quick Check 2

QUIZ

Your colleague just ran the model, looked at the test-set RMSE (got 0.025), tweaked some hyperparameters, and re-ran the test. What's the problem?

A

Nothing — running the model twice is harmless

B

The model is now overfit, so the new score is worse

C

The test set was used for tuning → score is biased

D

Hyperparameter tuning is never allowed for regressors

Pick A, B, C, or D — then advance.

Quick Check 2 — Answer

ANSWER

A

Nothing — harmless

Every peek at test contaminates it — even passive peeking counts.

B

Now overfit

Overfitting is a separate concept — the issue here is leakage.

C

Test set used for tuning



The test score is no longer an honest estimate of generalization.

D

Tuning not allowed

Of course it's allowed — but use the validation set.

The iron rule: the test set is a **one-shot**. Tune on the validation set (or CV inside the training pool). Touch the test set **exactly once**, for the final report.

Worked Example — Spot the Leak

WORKED
EXAMPLE

Below is your colleague's photo-z pipeline. Find the leak — there is exactly one.

```
df = pd.read_csv("photometry.csv")
y = df["z_spec"]
X = df.drop(columns=["z_spec"])

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Now split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

rf = RandomForestRegressor(n_estimators=100).fit(X_train, y_train)
print(f"Test RMSE: {rmse(y_test, rf.predict(X_test)):.4f}")
```

Pause and find the leak — then advance.

Worked Example — Solution

ANSWER

The leak:

`scaler.fit_transform(X)` was called on **the full dataset** before `train_test_split`. → the test set's μ and σ contributed to the scaling parameters applied to train.

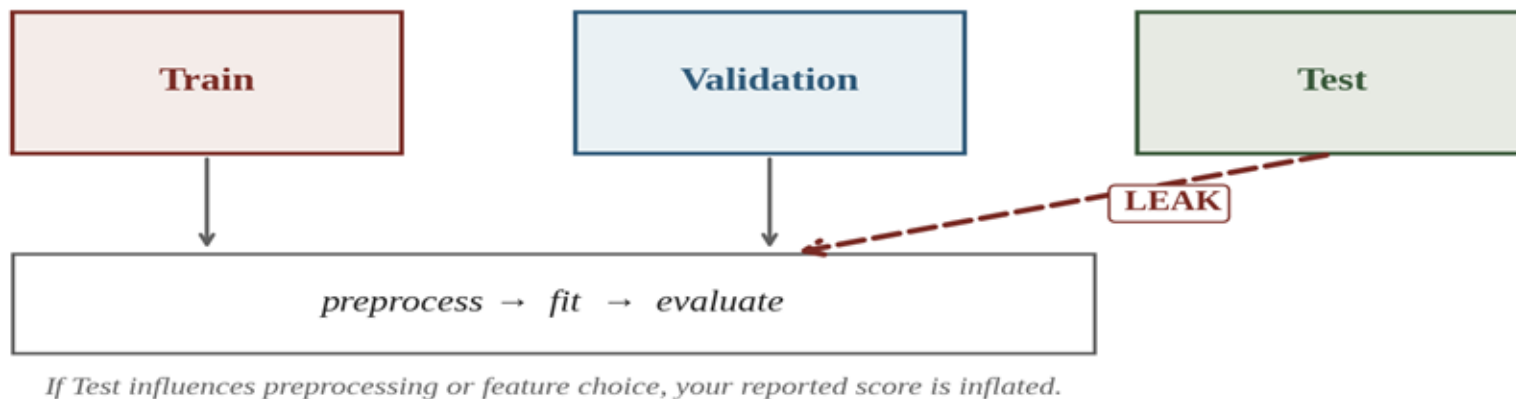
The fix — split FIRST, fit scaler on TRAIN only

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)    # fit on train only
X_test  = scaler.transform(X_test)         # apply same transform
```

Even better: wrap it in a **Pipeline**. The scaler is then automatically refit inside every CV fold — leakage becomes impossible.

Data Leakage



Definition: any information from the validation or test set that influences the trained model.

Symptom: your test-set score is suspiciously high, but the model flops on real new data.

Where Leakage Hides

1. Fitting preprocessing on all data

StandardScaler mean/std computed on X (train + test) before splitting — test stats leak in.

Fix: Fit transformers inside a Pipeline, on TRAIN only.

2. Target leakage in features

A feature secretly contains y — e.g. an ID encoding redshift, or a flag built using the answer.

Fix: Audit features. Drop anything derived from y.

3. Reusing the test set

Look at test → re-tune → 'test' again. Test set has been used for tuning.

Fix: Touch test ONCE. Use validation for tuning.

Code — Splitting Done Right

All splitting + preprocessing happens inside a Pipeline; never on raw X.

```
from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     GroupKFold, cross_val_score)

# 1. Carve off a final test set that you DO NOT TOUCH until report time
X_pool, X_test, y_pool, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=42)

# 2. Inside pool: K-fold CV for hyperparameter tuning
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipeline, X_pool, y_pool, cv=cv, scoring="f1")
print(f"CV F1 = {scores.mean():.3f} ± {scores.std():.3f}")

# 3. Final report: fit on the full pool, evaluate ONCE on test
pipeline.fit(X_pool, y_pool)
test_f1 = f1_score(y_test, pipeline.predict(X_test))
```

Recap — Reliable Evaluation

1

Split into Train / Val / Test

Carve off Test once. Touch only at the very end.

2

Use K-fold on the pool

Average over folds for a robust score; report mean $\pm$ std.

3

Stratify, group, time

Match the split structure to the structure of your data.

4

Audit labels

Spec-z vs photo-z; flag noisy or biased labels explicitly.

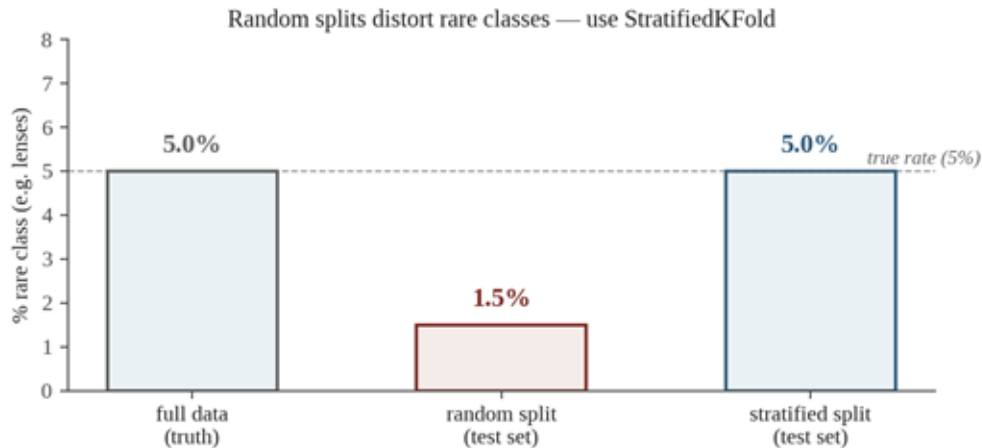
5

Guard against leakage

Preprocessing in Pipeline, no peeking, drop target-derived features.

Next → **Lecture 3b:** decision trees, bagging, and random forests.

When Plain K-Fold Isn't Enough



Stratified K-Fold

Keep class proportions equal across folds. Use when classes are imbalanced (lenses, quasars).

Group K-Fold

All examples from one group go to one fold. Use when objects share common properties.

Time-series split

Train on the past, validate on the future. Use for transients and time-domain surveys.

Ask of every dataset: rare class → `Stratified`; grouped → `GroupKFold`; time matters → `TimeSeriesSplit`.

Worked Example — CV Statistics

WORKED
EXAMPLE

Scenario

You ran **5-fold cross-validation** of a random-forest regressor and recorded the test RMSE per fold:

rmse = [0.026, 0.024, 0.027, 0.023, 0.025]

Report your model's performance as **mean $\pm$ σ** .

Compute on paper:

1. The mean RMSE across the 5 folds.
2. The standard deviation (use the $N - 1$ divisor).
3. Report as mean $\pm$ σ .

Pause and try — then advance.

Worked Example — Solution

ANSWER

Step 1 — Mean

```
sum  = 0.026 + 0.024 + 0.027 + 0.023 + 0.025 = 0.125
mean = 0.125 / 5 = 0.0250
```

Step 2 — Standard deviation

```
diffs from mean: +0.001, -0.001, +0.002, -0.002, 0
squared:          1e-6,  1e-6,  4e-6,  4e-6,  0
sum of squares   = 10e-6
variance (N - 1) = 10e-6 / 4 = 2.5e-6
σ = √(2.5e-6) ≈ 0.0016
```

Report: RMSE = 0.0250 ± 0.0016

Why both numbers matter: the mean tells you accuracy. The σ tells you *stability* across data splits — a big σ signals the score is sensitive to which examples you happened to hold out.